

Algoritmer och Datastrukturer

Föreläsning 11

Test-Driven
Development



Mål för dagen

- Lära oss hur man skapar en enkel testmiljö i IntelliJ med Maven
- Veta vad ett enhetstest är
- Förstå hur vi skriver test och varför
- Lära oss principerna bakom Test-Driven Development
- Jobba med ett praktiskt exempel på hur man kan bygga upp en algoritm långsamt från grunden

Varför vi behöver lära oss att koda cleant

- Ren kod gör det **lättare att tänka och resonera kring funktionalitet**
- Det gör den **lättare att följa och reducerar behovet av kommentarer**
- Förklarande **inlinekommentarer** bör ses som **ett misslyckande** i bra kod: om vi plötsligt behöver förklara vad en bit kod gör är det ett tecken på att den inte längre gör någonting som tillhör den nuvarande metoden, och den här logiken bör då brytas ut till ett separat metodanrop i stället
- Samma med variabler: i stället för att skriva kommentarer som berättar vad en variabel är, **döp den till något som är självförklarande!**
- När vi enkelt kan se vad kod gör är det också **lättare att fundera över om den gör rätt saker**, och om den försöker lösa rätt problem

Naiva lösningar

- Ser till att vi har någonting som fungerar för stunden
- Låter oss skapa en grund som vi sedan kan bygga vidare på
- Program är komplexa, men de börjar inte så: precis som en bok startar de med en tom sida, och sedan börjar vi strukturera upp innehållet
- Om vi inte börjar i den här änden kommer vi att misslyckas med vad vi försöker ta oss för

Vikten av naiva lösningar

- På inledande Java brukar vi fråga: “Kan ni skriva kod som kollar om en samling innehåller unika nummer?”

```
int[] numbers = { 10, 12, 3, 4, 5, 5, 8, 6, 12 };
```

```
public boolean containsDuplicates(int[] numbers)
{
    for (int i = 0; i < numbers.length; i++)
    {
        for (int j = i + 1; j < numbers.length; j++)
        {
            if (numbers[i] == numbers[j]) { return true; }
        }
    }

    return false;
}
```

- Ok naiv lösning som funkar out of the box
- För varje n kollar vi alla andra n
- Tidskomplexitet: $O(n^2)$
- Lätt att skriva, men inte jätteläsbar
- Kan vi refaktorera och göra koden tydligare?

Vikten av naiva lösningar

- for-looparna gör inte samma sak! Vi har en metod som egentligen gör två saker: dela upp den i två i stället för tydlighetens skull

```
public boolean containsDuplicates(int[] numbers)
{
    for (int number : numbers)
    {
        if (!isUnique(numbers, number)) { return true; }
    }

    return false;
}
```

```
private boolean isUnique(int[] numbers, int n)
{
    int count = 0;

    for (int number : numbers)
    {
        if (number == n) { count++; }
        if (count > 1) { return false; }
    }

    return true;
}
```

- `containsDuplicates()` är lättare att läsa och förstå nu
- “For each number in numbers: If not unique ... return true” ser nästan ut som engelska
- **Fortfarande $O(n^2)$** : vi har bara flyttat den nästlade for-loopen till en egen metod
- Är “Är det här talet unikt” verkligen rätt fråga att ställa? Kan vi omformulera problemet?

Vikten av naiva lösningar

- I stället för “Är det här talet unikt?” kan vi ändra frågan till “Har jag redan sett det här talet?”

```
public boolean containsDuplicates(int[] numbers)
{
    Set<Integer> visitedNums = new HashSet<>();

    for (int number : numbers)
    {
        if (!visitedNums.add(number)) { return true; }
    }

    return false;
}
```

- **$O(n)$ tid!**

- HashSet har konstant tid på insättning (i genomsnitt)
- `add()` returnerar false om vi försöker stoppa in ett värde som redan finns
- Att göra problemet tydligare innebär att **vi kan resonera och tänka tydligare kring det**
- Kort och ren lösning, men den **måste evolvera fram inkrementellt**
- Vi ska prata om **hur** i dag

Hur vi utvecklar kod inkrementellt

- Ett sätt att utveckla kod inkrementellt på, som vi kommer att intressera oss för i dag, är genom Test-Driven Development
- **Test-Driven Development går ut på att låta testningen driva själva utvecklingen**
- Skiljer sig från **vanlig testning** där man normalt testar kod efter att den är designad för att kolla att den fungerar: **testet existerar vid sådana tillfällen för att validera designen**
- **Test-Driven Development är inte en testmetod: det är ett sätt att designa mjukvara från grunden**
- Vi ska prata mer om TDD om en stund. Vi ska dock börja med att prata lite om vad tester är för någonting, och vilka fundamentala problem och risker som finns i alla system, innan vi kikar på hur TDD försöker adressera dem

Vad är unit testing?

- Ett **enhetstest** (“unit test” på engelska) är ett test som är skrivet för att testa en isolerad del av ett program, för att bekräfta att denna del fungerar som vi förväntar oss
- Det finns även integrationstester, systemtester, GUI-tester, osv
- En **enhet definieras som ett enskilt beteende inom ett system**: kan vara en metod men ibland också en klass (eller, mindre vanligt, en modul)
- **Idén är isolering**: enheten ska testas självständigt från databaser, filsystem, API:n, osv
- Om det finns sådana externa beroenden brukar man ersätta dem med mockdata

Ett korrekt enhetstest bör:

- * Bara testa en sak
- * Vara snabbt
- * Köras i isolering
- * Vara upprepningsbart
- * Inte vara beroende av externa system



Tester validerar kod åt oss automatiskt

- **Enhetstestning är så kallad automattestning** (dvs sker inte manuellt)
- Ni har allihop vid något tillfälle gjort metoder eller klasser som innehåller klasser eller main()-metoder för att kunna testa er kod så att den gör rätt saker
- Problemet är att **det här snabbt blir ineffektivt när program växer i storlek och komplexitet**: Vi behöver ständigt ha koll på vad som ändras, och varje gång måste vi köra alla våra testklasser och kontrollera att programmen beter sig som de ska
- Sådan här **manuell testning är bra för att utvärdera hur ett program är att interagera med** (bra att göra regelbundet för spel och interaktiva program som måste “kännas rätt” och ha logiska workflow), men det är ett dåligt och ineffektivt sätt att validera att koden fungerar korrekt
- Enhetstestning är ett sätt att automatisera den här processen med små snabba tester, och vi kan sedan köra dem tusen gånger gratis

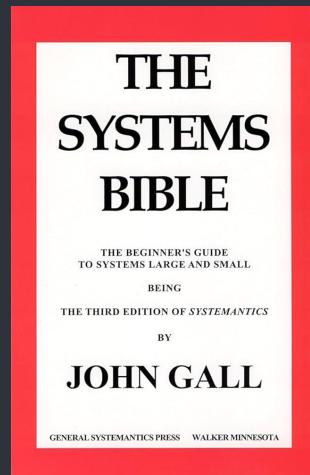
Fördelar med unit testing

- Enhetstester fångar upp buggar och fel så fort du skriver koden - inte flera veckor senare när saker går sönder under integration
- Du vet direkt om du klantat till det, och kan i så fall fixa problemet
- System och program är komplexa, och ju mer komplexa de är desto fler saker finns det som interagerar med varandra eller bygger på varandra
- Att ändra på ett ställe i ett program påverkar ofta flera andra ställen på ett vis som är svårt att förutspå på förhand
- Enhetstester blir vår garanti för att koden fortfarande fungerar korrekt

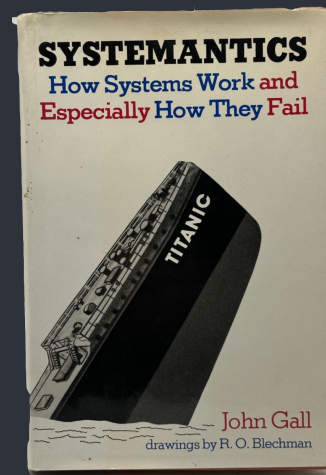


Systemantics: om varför system misslyckas

- “Systemantics” är en sammanslagning av orden system och semantik
- Skriven av John Gall, som egentligen inte pratade specifikt om IT-system: första upplagen är från 60-talet
- “Allting är ett system” – generella lärdomar för all form av systemtänkande och -design
- Några av de axiom som boken grundar sig i:
 - * “Systems in general work poorly or not at all”
 - * “Systems tend to oppose to their own proper function”
 - * “A complex system can fail in an infinite number of ways”
 - * “As systems grow in size, they tend to lose basic functions”



Moderna upplagan



Första upplagan

“A complex system that works is invariably found to have evolved from a simple system that works.

A complex system designed from scratch never works and cannot be patched up to make it work.”

**“Gall’s Law”,
från Systemantics**

Komplexa IT-haverier

Polisen lägger ned kritiserat IT-system

UPPDATERAD 21 FEBRUARI 2014 PUBLICERAD 20 FEBRUARI 2014

Polisen lägger ner det starkt kritiserade IT-systemet Pust Siebel, som används för att rapportera in brott.

Nu ska Rikspolisstyrelsen ta fram en avvecklingsplan, enligt ett pressmeddelande.

Systemet har fått hård kritik, och har dragits med stora problem. Poliser har larmat om att det tagit timmar att rapportera in enkla brott.

Tieto stor vinnare på it-fiasko för 1,5 miljarder

Skolplattformen, Sveriges mest utskälda it-projekt, är skrotat och minst 1,2 miljarder skattekronor är borta.

”De enda som har tjänat på det här är konsulterna”, säger skribenten Olle Lidbom som nu släpper boken ”Skolplattformen – en sann historia om miljarder, konsulter, föräldrar och barn”.

Millennium togs i drift i [Västra Götalandsregionens](#) södra område den 12 november 2024. Tre dagar senare hade det pausats i ren panik. Ingenting fungerade som det skulle och personalen var förtvivlad och traumatiserad.

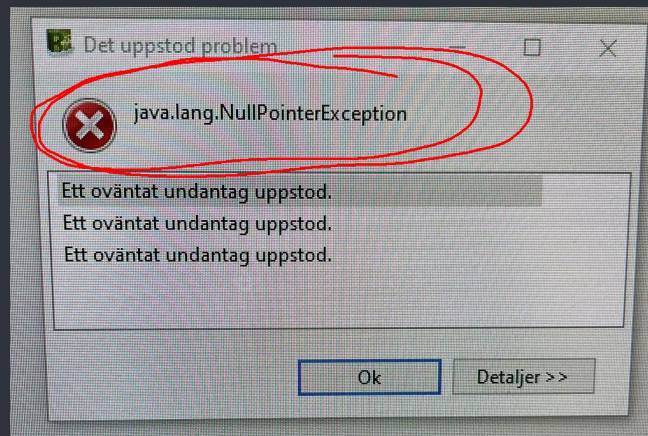
Mer än 6 miljarder

Prislappen? 5,5 miljarder kronor för ett system som inte går att använda.

Utöver det har regionen betalat [konsultkostnader för 799 miljoner kronor](#). Trots att införandet av [Millennium har pausats](#), fortsätter flera av dessa konsulter att arbeta, vilket medför ytterligare kostnader varje månad.

Millennium: Sveriges största IT-fiasko genom tiderna

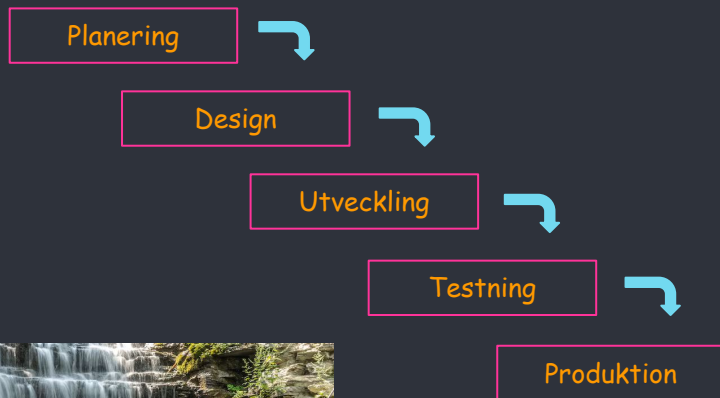
- Oracle sålde ett IT-system till svensk sjukvård
- **Döpt till Millennium eftersom det skapades innan sekelskiftet:** man köpte alltså 25 år gammal kodskuld
- **Anpassat för amerikansk sjukvård** som är försäkringsbaserad: man tänkte sig att systemet skulle modifieras för Sverige
- Gammal knakig Javakod: omoderna koncept, NullPointerExceptions överallt, delarna hängde inte samman
- Systemet **köptes in färdigt i stället för att byggas upp från grunden**
- I stället för att utgå från de behov sjukvården hade försökte man skapa rutiner som vårdpersonalen skulle följa, vilket **resulterade i mycket mer jobb** för dem - i de få fall där systemet faktiskt fungerade
- Var i drift i två dagar; **5,5 miljarder skattekronor i sjön**



“A complex system that works is invariably found to have evolved from a simple system that works.”

Systemutvecklingsmetoder

Klassisk utvecklingsmodell: varje steg görs färdigt innan nästa påbörjas (rinner in i varandra som ett vattenfall)



Precis som ett vattenfall går den bara i en riktning: rinner aldrig uppåt

Modern utvecklingsmodell: små cykler som upprepas om och om igen (vi itererar tillbaka till tidigare steg för att förbättra och förtydliga) - modellen loopar



Iterativ problemlösning genom TDD

- Den fundamentala principen bakom agil utveckling var att **ett fungerande program är bättre än inget program alls**
- En fungerande algoritm är (för det mesta!) bättre än ingen algoritm alls enligt samma devis
- Vi har pratat lite om **naiva algoritmer** (den simplaste lösningen på ett problem) **och brute force** (testa alla möjliga kombinationer tills vi hittar en som fungerar) på kursen, och även om vi gärna vill undvika kod som fungerar så här i ett färdigt program kan de ändå utgöra en bra startpunkt för oss
- När vi har kod som fungerar är det lättare att **refaktorera den så att den blir mer effektiv**
- Test-Driven Development argumenterar för just det här: tänk inte för långt i förväg i början. Komplexiteten får komma senare
- **Om en komplex alrogitm ska ha en chans att fungera måste den först evolvera från en simplare algoritm**

Test-Driven Development



- I Test-Driven Development låter vi testen driva utvecklingen
- Det här innebär att vi börjar med att skriva ett test innan det finns någon kod alls! Vårt första mål är sedan att göra minsta möjliga för att få det här testet att lyckas

Red-Green-Refactor

- **Rött** betyder: vi skriver ett test som misslyckas
- **Grönt** betyder: vi ser till att testet går genom med minsta möjliga ansträngning
- **Refaktorerar** koden efter det: se till att bygga upp logiken så att den kan generaliseras och samma kod kan användas till flera olika testfall; fundera på om den går att optimera, osv

Skeptisk
student



“Men ååh, asså vadå, skriver du verkligen alltid testet först när du vet att det failar?? Ehh o_0”

- Ja, när man jobbar med testutveckling gör man det. **Det misslyckade testet blir ett kvitto på var någonstans man är och vad man behöver göra härnäst**
- **Extremt värdefullt** eftersom det visar att testet kan fånga upp ett fel
- Alla program behöver inte test-driven development, men de som gör det blir mycket bättre av att vi följer de här tre stegen korrekt, även om det ibland kan kännas onödigt

Test-Driven Development

- Kent Beck definierade den här metodiken i början av 00-talet, men den har funnits mycket längre (erfarna programmerare hittade hit naturligt)
- Beck är även en av de ursprungliga 17 undertecknarna av det Agila Manifestet

- **TDD och agil utveckling är nära associerade med varandra**

- Poängen med TDD är att undvika förhastad optimering ("premature optimization" på engelska)

- Vi låter testen driva utvecklingen i stället för att bara testa hur robust själva lösningen är: att ta oss förbi ett test är ett krav för att kunna börja arbeta på nästa test



- **Inga tidigare test får gå sönder** när vi förändrar systemet (i så fall: fixa)
- Vi utvecklar i **små inkrementella steg**, vilket är det **enda sunda sättet att utveckla program på**: notera hur vi även gör atomära commits, skriver korta metoder, försöker få dem att bara göra en sak åt gången, osv

Varför ständig testning är viktigt

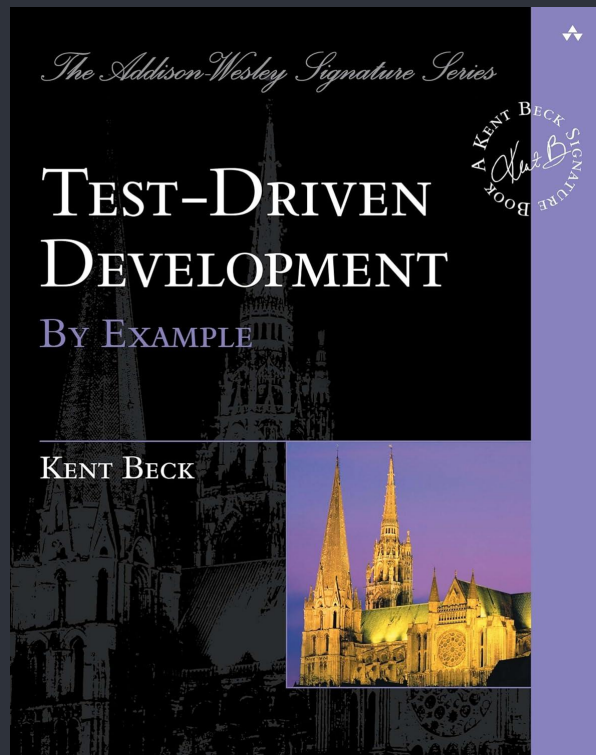
- Den centrala idén bakom Test-Driven Development är att mjukvara behöver evolvera gradvis, och **den behöver framför allt fungera genom varje steg i utvecklingen**
- När vi jobbar med TDD utsätter vi mjukvaran för konstanta test för att se om den fortfarande fungerar: vi tvingar den att kämpa för sin existens
- Vi gör det här för att **kunna fånga upp oväntade mutationer så fort som möjligt**
- När vi refaktorerar bör ALLA tidigare test lyckas: om något av dem misslyckas måste vi åtgärda det direkt, med minsta möjliga ansträngning
- Kom ihåg Galls axiom:
 - * **“Systems tend to oppose to their own proper function”**
 - * **“A complex system can fail in an infinite number of ways”**



Murphys lag: allting som kan gå fel kommer att gå fel

Bra boktips

- Referensverket för TDD: bästa boken om ämnet
- Använder Smalltalk för vissa kodexempel och Java för andra, men är väldigt generell och går utmärkt att läsa oavsett vilket språk man känner sig mest hemma i
- Bra fördjupning om man är intresserad av TDD
- Vi kommer bara skrapa lite på ytan i dag vad gäller Test-Driven Design, men min förhoppning är att ni kommer intressera er mer för det här på egen hand, för det är en extremt värdefull metodik om man vill bli en bättre programmerare
- Man behöver inte följa den slaviskt, men om ni försöker använda huvudprinciperna när det finns ett behov av att utveckla inkrementellt kommer ni automatiskt bli dubbelt så bra utvecklare



kaffepaus(15);



Att komma igång med unit testing

- För enhetstestning använder vi ett tredjepartsbibliotek som heter JUnit 5
- Femte generationen av ett populärt framework, ursprungligen skapat av Kent Beck på 90-talet
- Det finns legacyversioner JUnit 2, 3, osv men dessa används bara om man jobbar med gamla kodbaser (pre Java 8)
- JUnit går att lägga till manuellt i ett projekt men det är ganska krångligt: när vi arbetar med kod som vi behöver hämta in någon annanstans ifrån brukar vi därför använda något som kallas för en dependency manager
- Det finns två populära i Java: Maven och Gradle. Vi kommer kolla på Maven i dag eftersom det är lite slimmare och bättre anpassat för våra behov, men om någon redan är bekant med Gradle får ni självklart använda det i stället: jag kommer bara inte visa er hur det fungerar
- Låt oss öppna IntelliJ och sätta upp ett testprojekt!



Lunch(60);

Vi ses igen i E31 13.15

Kodkata: Romerska Nummer

- Finns många varianter av den här övningen: jag gjorde en TDD-baserad version där man ska översätta decimaltal till romerska tal: matar vi in 4 ska metoden spotta ut IV osv
- Jobba själva, i par eller i grupp: ingen examinerande uppgift så ni behöver inte lämna in någonting
- Mot slutet går vi genom resultaten litegrann och kollar med varandra hur långt folk kommit, vilka lösningar som producerats, vad som var lätt och svårt, osv
- Tänk alltid Red-Green-Refactor, och refaktorera inte för tidigt
- Arbeta i så små inkrementella steg som möjligt

Kata: Konvertera romerska nummer

Vad är: En kata är en programmeringsövning som skrivs under en sitning för att öva upp färdigheter i kodskrivande och logiskt tänkande. Ordet är lånat från japanskan där det syftar till kampsportskoreografi. Det norpades på oklara grunder av Dave Thomas när han skrev boken *The Pragmatic Programmer* (1999). Eftersom det lät mystiskt och coolt fick det snabbt fäste bland programmerare (som ofta är mkt ocool och är i stort behov av mer coolhet).

Problembeskrivning

Katan är en känd programmeringsövning som går ut på att skriva en metod för att konvertera decimalnummer till romerska nummer. Romerska nummer använder kombinationer av bokstäver från det latinska alfabetet för att beteckna värden. De baseras på följande sju symboler:

I	1
V	5
X	10
L	50
C	100
D	500
M	1000

Nummer formas genom att kombinera symboler och addera deras värden. Oftast placeras symboler i enlighet med de värden de representerar: de börjar med största värdet först. (Precis som vi i decimal sätter den största siffran längst till vänster: 1111 i bas-10 betyder $1000 + 100 + 10 + 1$).

När mindre värden står skrivna framför större värden i en romersk sekvens innebär det att det mindre värdet ska subtraheras från det större: IX betyder exempelvis 9 eftersom $X - I = 10 - 1$.

Nummer	Beräkning	Värde	Kommentar
III	$1 + 1 + 1$	3	Enbart addition utförs
DCX	$500 + 100 + 10$	610	Enbart addition
DCIX	$500 + 100 + (10 - 1)$	609	Addition och subtraktion

Uppgift

Skapa en algoritm som konverterar decimaltal till romerska nummer genom Test-Driven Development. Algoritmen behöver bara kunna hantera decimaltal upp till tusen (M). Ni får jobba ensamma eller i grupper om två eller flera.

Kodkata: Romerska Nummer

- Fastnar ni så fråga mig eller Lovisa, eller andra studenter, eller Mr. Roboto
- Ägna max 10 min åt varje testfall: det är viktigare att gå vidare och se hur algoritmen tar form än att lösa allting själv
- Publicerar lösningsförslag i studium, men använd bara som hjälp för ett steg åt gången
- Försök fundera själva, och försök att inte optimera för mycket i förväg - programmet kommer bli bättre i slutändan

▼ Vecka 9 Test-Driven Development samt eget arbete med projektet	
Om vecka 9	
Dugga 2	1 mar 100 poäng
Test-Driven Development.pdf	
Romerska_Nummer_Kata.pdf	
Romerska_Nummer_Lösningsförslag.pdf	